3D Geographic Scenes Visualization Based on WebGL

Ru Miao[a]
[a] School of Computer and Information Engineering, School of Economics Postdoctoral Research Station
Henan University
Kaifeng, China
mr1015@henu.edu.cn

Jia Song[b,c]
[b] State Key Laboratory of Resources and Environmental Information System, Institute of Geographical Sciences and Natural Resources Research, Chinese Academy of Sciences
Beijing, China
[c] Jiangsu Center for Collaborative Innovation in Geographical Information Resource Development and Application
Nanjing Normal University
Nanjing, China
* Corresponding author:
songj@lreis.ac.cn
Tel.: +86-10-6488-9906

Yunqiang Zhu[b,d]
[b] State Key Laboratory of Resources and Environmental Information System, Institute of Geographical Sciences and Natural Resources Research, Chinese Academy of Sciences
Beijing, China
[d] Collaborative Innovation Centre for Baiyangdian Basin Ecological Protection and Jingjinji Regional Sustainable Development
Hebei University
Baoding, China;
zhuyq@igsnrr.ac.cn

***Abstract*—With the fast development of the Internet, the lightweight 3-dimension (3D) geographic scenes visualization system is expected to play more important roles in web-based GIS. Finding an effective solution to render 3D geospatial data has great significance in Cartography and Geographic Information Systems. The emergence of HTML5, as well as WebGL (Web Graphics Library), provides a new approach for 3D WebGIS. WebGL is a JavaScript API for rendering 3D graphics within compatible web browsers. Other plugins are no longer required for displaying 3D objects. WebGL is regarded as a part of new generation web standards and is completely integrated into most popular browsers. It allows GPU accelerated usage of physics and image processing. WebGL elements can be mixed with other HTML elements and composited with other parts of the page or page background. This paper discusses the design and implementation of a digital city roaming system based on WebGL technology. Users can make cartographic analysis, destination query, data query and map mark with the system in browsers. All 3D models in the scene are converted to glTF (GL Transmission Format), and can be used to load 3D scenes and models. For the complex scenes, this system only downloads the scene data within the current view using visibility culling techniques, then joins these data into the rendering queues. At the same time, the data request task is managed in queue by the management layer using prefetching strategy. On the client-side, a method is proposed to deal with occlusion culling based on the parameters of current scene, geometry shape and positional relation. The client-side removes the occluded model data from memory, while the remaining visible data is passed to the GPU rendering pipeline. Then the client-side renders the scene in browser according to the parameters of these glTF files. The experiment result shows that this system can smoothly render the city 3D scene in the browser without any plugins. The city roaming system based on WebGL shows a good interactive experience of 3D visualization for users. The paper provides a very effective and promising method for establishing 3D WebGIS realization.**

Keywords—WebGIS; WebGL; 3D visualization; occlusion culling; city roaming

I. INTRODUCTION

The emergence of Web3D technology has injected new vigor into 3D graphics fields. Now, Web3D technology mainly includes VRML, X3D, Viewpoint, Java3D, and Shockwave3D; the combination of Java3D with VRML [1] and GeoVRML [2] is the main one. The emergence of HTML5, as well as WebGL, provides a new approach for 3D WebGIS [3]. WebGL is a JavaScript API [4-6] and is a cross-platform, royalty-free web standard for low-level 3D graphics API based on OpenGL ES 2.0. WebGL elements can be mixed with other HTML elements and composited with other parts of the page or page background. In Internet Explorer, WebGL can currently only be used through plugins, and the model data and the core part of system functions are concentrated in the server side. Additionally, the client side can only install a browser that can do simple interactions, such as moving, scaling, and querying. WebGL is widely supported by mainstream hardware products and mobile web browsers like Mozilla Firefox, Google Chrome, Safari, and Opera. With the rapid development of Web3D technology, various kinds of Browser/Server (B/S) 3D scene models also come up.

In the past, most software did not have the capability of loading large-scale urban 3D models, and some serious hysteresis phenomena appeared when browsing these large-

scale scenes. Also, 3D city scenes have a great amount of data, including tens of thousands of buildings. Among these buildings, a large amount of data information (such as texture, lighting, and coordinates) is also needed in drawing and rendering the virtual scene. Users need a long time to download these city scenes, the model data of the whole scene cannot be called into the system memory at any single given moment, and the GPU cannot render in real-time. Therefore, the primary problem of 3D geographic scenes visualization is to solve the transmission bottleneck caused by network bandwidth. At present, there are some solutions: the simplification of 3D models, the compression of model data, the prefetching strategy, and others. Schroeder [7], Isler [8], and Hoppe [9] proposed a variety of ways to simplify 3D models and generate streaming coding. The streaming encoding technology enables users to reconstruct a rough 3D scene even with less model data and also accelerates the client's response speed.

The transmission bottleneck by the bandwidth is eased to a certain degree by these methods. However, the streaming coding generated in the simplification process is not very compatible with the current 3D scenes visualization mechanism, and it is impossible to unify the data formats provided by different models. At the same time, there are no generic interfaces to parse and show the model, and significant time is wasted before rendering the scene. The system must also install some plugins before running, and the configuration process is complex and has poor portability. In addition, simplification and compression of 3D models reduces the data volume to a certain extent, but they may not be reduced enough to satisfy the client response. Due to the scope of vision and object occlusion, users can only see parts of the scene during a certain time. Therefore, when the user is roaming, if we can determine the next scenes that the users might see, then these scenes can be downloaded to the client side in advance. This would greatly reduce the response time, and improve the speed of loaning in complex scenes, and thus meet the demand of showing scenes in real time.

According to the above problem, in this paper, we studied the architecture of Web-based 3D city model scenes, the 3D model formats and scene organizations, and 3D scene scheduling methods. We then designed and implemented a digital city roaming system based on WebGL technology. The experiment results showed a good interactive experience of 3D visualization for users based on WebGL in the browser, without any plugins.

II. METHODOLOGY

Aimed at the problems in large-scale 3D scene visualization, a framework fully considering the efficiency of the visualization was designed in this paper, as shown in Fig. 1.

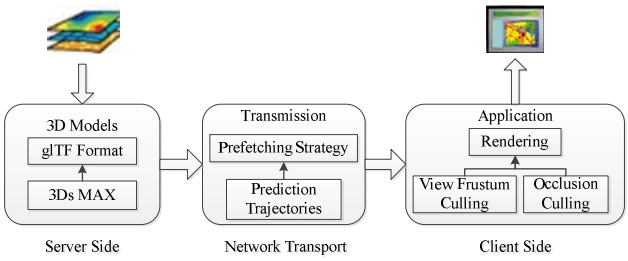


Fig. 1. A framework for 3D geographic scenes visualization.

In the framework, the web server side mainly provides data services and stores the 3D scene data in g1TF format. The large-scale complex 3D scene of the city was based on the actual collected city data by 3Ds Max software. Then, all 3D models in the scene were converted to glTF (GL Transmission Format, a runtime 3D resources format). The network transport layer was mainly responsible for scheduling scene data using the prefetching strategy based on prediction trajectories. Finally, the client side rendered the 3D scene efficiently using occlusion culling technology. View frustum culling and occlusion culling technology based on the parameters of the current scene, geometry shape, and positional relations, were proposed to deal with occlusion culling to accelerate rendering.

A. Construction and Organization of 3D Scenes

When browsing 3D city scenes on the fly, 3D building model data is one of the main types of data in the network transmission. Since the amount of scene data, network transmission mode, and client group scheduling policy will be directly affected by the structure of the model data, a general 3D model format with little space occupation, rapid transmission, and good graphical interface is needed.

The glTF designed especially for the WebGL interface was used in this paper [10]. The structure of the glTF format is shown in Fig. 2. It is mainly divided into 4 parts. 1. JSON file (.gltf). This is the core of the entire model, and stores the nodes level, material quality, cameras, and lighting parameters of the model. 2. Binary file (.bin). This is used for storing models of graphic data, such as vertex coordinates, texture coordinates, indices, animation, and so on. 3. Image file (.png, .jpg, etc.). This is used for the texture model. 4. Shader file (.glsl). This includes the vertex shader and fragment shader, which are required for graphics rendering.

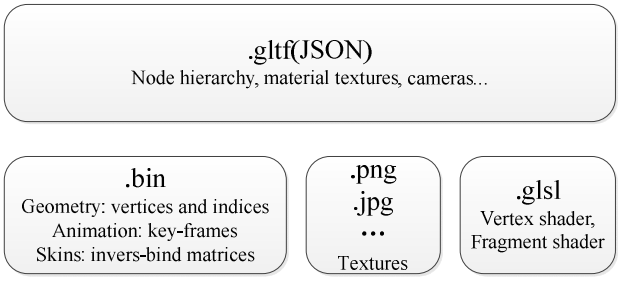


Fig. 2. glTF format.

The main advantages of using glTF to build 3D scenes are as follows:

This study was supported by the National Natural Science Foundation of China (No. 41501432; No. 41201411); National Special Program on Basic Works for Science and Technology of China (No. 2013FY110900); Foundation of State Key Laboratory of Resources and Environmental Information System (No. O88RA20CYA).

Firstly, the data compression is reasonable, and binary files and JSON data files, which occupy little space, are used for storage.

Secondly, glTF defines an extensible, common publishing format for 3D content tools and services. It uses a JSON format to store the main information of the model. This format is suitable for Web transmission and convenient for JS parsing. At the same time, the image format supported by JS is used as the texture data, such as JPG, PNG, etc.

Thirdly, glTF offers full rendering optimization. The model contains the GLSL shader, which can be applied to the GL interface directly. Meanwhile, the binary file is used for storing graphic data, and the data buffer can be created conveniently and quickly.

Finally, glTF is well connected to the GL interface. The property parameters of the model can be mapped to the functions and parameters of the GL interface easily.

In this paper, we used the glTF file to organize the 3D model data efficiently. The glTF file recorded the ID, name, latitude and longitude, orientation, scale, and model links of all models in the scene. The model links are shown by the uniform resource locator (URL, the standard resource address on Internet), which is the storage location on the web server.

The specific process of building the city scene is shown in Fig. 3.

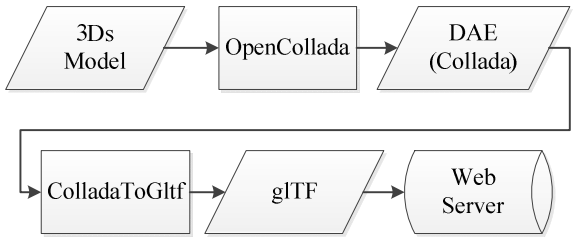


Fig. 3. Construction flowchart of a 3D scene.

In Step 1, we used a 3D Studio Max, which is a 3D modeling and rendering software, to make large-scale 3D city scenes based on the actual city data.

In Step 2, these 3D models were exported in COLLADA format using the third-party format plugin OpenCollada.

Then, in Step 3, all COLLADA models in the scene were converted to the glTF format by the transformation tool provided in an open source “cesium” project.

Finally, in Step 4, these scene data were uploaded to the web server according to the storage location of the scene data described in the glTF files.

B. Prefetching Strategy Based on Prediction Trajectories

For 3D virtual scenes, the data volume is growing in intensity. However, users can only see part of a scene within a certain time due to the scope of their vision and object occlusions. Therefore, when a user is roaming, the system should download the local scene automatically by determining the next scenes that user may be interested in. It would greatly reduce the response time and improve the flow of scenes.

When roaming, there is a certain pattern of changes of the user's viewpoint. Therefore, we can download some scenes in advance by predicting user trajectory. Predicting the trajectory of users is based on the user's browsing history. The client records trajectory coordinates by monitoring the user's keyboard response events. Then, the average of historical trajectories is calculated, and the next scenes are predicted by recording the location of the view movement and sending this data to the server side (Fig. 4).

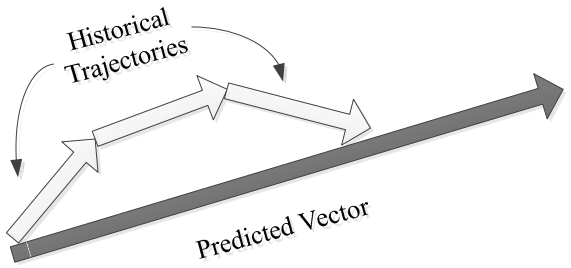


Fig. 4. Trajectory prediction method.

First, a historical trajectory is a set of points made up of track points, which can be represented as Tra $\{lc_1, lc_2, lc_3\ldots\}$. Each track point in the historical trajectory $lc_i \in Tra$ can be expressed in a triad: $lc_i = (lat, lng, t)$. lat and lng respectively represent the latitude and longitude of the moving object at sampling time t.

Next, the client completes the recording of the trajectory coordinates by monitoring the user's keyboard response events.

Then, a linear function of time is used to represent the position and time of moving objects. For example, if the position of the moving object is recorded as lc at a given time tc, and the speed is called vc, then the position of this object after a period of time h can be calculated by a linear formula, as in (1):

$$position(h) = lc + vc \times (h - tc). \quad (1)$$

According to (1), we can then obtain a series of position information, Pre $\{p_1, p_2, p_3\ldots\}$, which is called predicted vector.

Finally, the server generates a pre-download scenes queue based on the user's viewpoint information and the predicating vector coordinates.

These scenes are pre-downloaded to the client before the client makes a request to the server. This strategy of downloading and rendering can improve the rendering speed.

C. Occlusion Culling Technology

After downloading the model data to the client using the prefetching strategy based on prediction trajectories, view frustum culling and occlusion culling technology were used to improve rendering efficiency.

View frustum culling eliminates objects that are not in the current view frustum, and only loads into memory objects that

are in the current view frustum [11]. This method relies directly on the observed parameters returned by the current system. Meanwhile, occlusion culling eliminates objects or parts of the objects that are invisible to users due to occlusion judging [12]. When watching a 3D scene, the occlusion phenomenon between ground objects is prominent. Actually, most objects in the view frustum are invisible in the current position, and the data of these objects do not need to be rendered into memory. This can further reduce the scope of loaded objects.

All scenes in the current visual field were projected to generate the view frustum. Then, objects not in the view frustum were eliminated, as shown in Fig. 5. In this figure, the dotted portions are eliminated objects or invisible surfaces of objects.

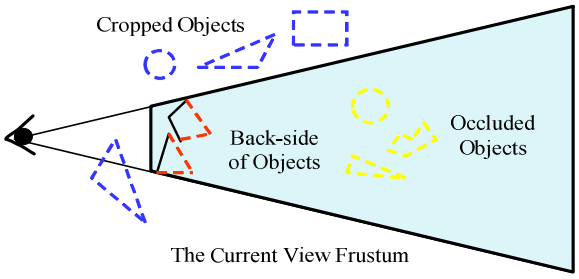


Fig. 5. View frustum culling and occlusion culling.

The eliminated objects are usually thought to be of 2 kinds: those out of the view frustum (blue objects), and those occluded (red and yellow objects). For the blue objects, we eliminated them using the view frustum culling geometric algorithm [13].

The view frustum consists of 6 sides: the top, bottom, left, right, near, and far. First, we determined the scope of the view frustum according to the current camera parameters, and established the equation of the 6 sides of the view frustum. The points A_1, A_2, A_3, A_4, P_b, and P_t are known, as shown in Fig. 6. The view frustum is identified by these points.

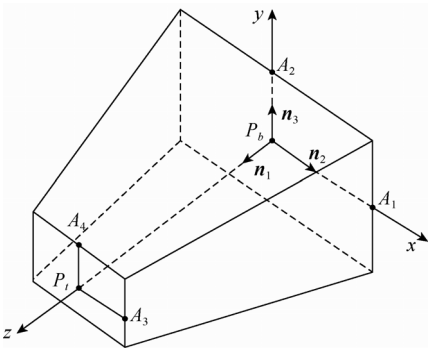


Fig. 6. Coordinates for view frustum culling.

Then, the minimum cube boxes of the models in the scene sets are obtained, and the bounding box is the external cuboid surrounding the models.

Next, we calculated the intersection between bounding boxes and the current view frustum. From this, we obtained different positions relative to the view frustum: separation, collision, intersection, and inclusion. Models that were separation and collision relative to the view frustum were eliminated.

Models that were intersection and inclusion of the view frustum need to be occlusion culled according to visibility judgment. The invisible faces of the objects were eliminated. For a convex polyhedron, the visibility judgment of surfaces can be based on the angle (θ) of the outer normal vector (N) of the surface and the visual vector (V) of users, as shown in Fig. 7. If the angle between the 2 vectors is greater than or equal to 0 degrees and less than or equal to 90 degrees, the surface is visible. If the angle is greater than 90 degrees and less than or equal to 180 degrees, the surface is not visible.

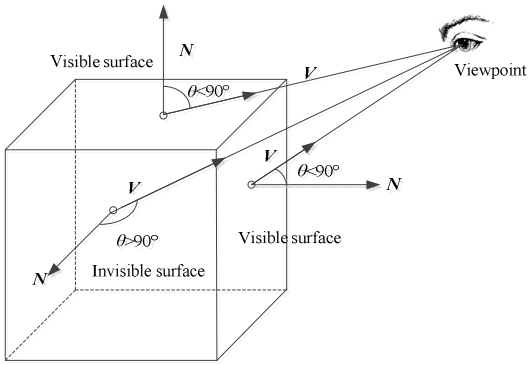


Fig. 7. Visibility judgment of object surfaces.

For each model in the view frustum, we first calculated the outer normal vector $\boldsymbol{N_i}$ of every surface according to the vertex coordinates.

Then, for each surface P_i of the model, we calculated the visual vector $\boldsymbol{V_i}$ according to the viewpoint's coordinates.

Next, we calculated the cosine of angle θ_i of the $\boldsymbol{N_i}$ and $\boldsymbol{V_i}$ as in (2):

$$\cos\theta_i = \frac{\boldsymbol{N_i} \bullet \boldsymbol{V_i}}{|\boldsymbol{N_i}||\boldsymbol{V_i}|} \quad (2)$$

If $\cos\theta_i \geq 0$, that is, when $0 \leq \theta_i \leq \pi/2$, the surface P_i faces forward and is visible. If $\cos\theta_i < 0$, that is, when $\pi/2 < \theta_i \leq \pi$, the surface P_i faces toward the back and is invisible.

Finally, we marked all visible surfaces, and put the scene data after occlusion culling into the rendering pipeline.

III. Application Case

A. Architecture of the 3D Roaming System

Aimed to tackle the problems of large-scale 3D scene visualization, a B/S mode system service architecture fully considering visual efficiency was designed, as shown in Fig. 8.

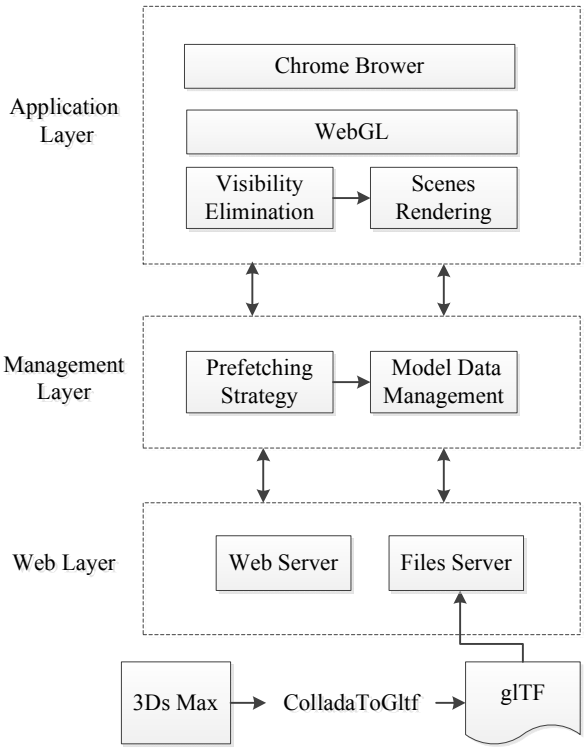


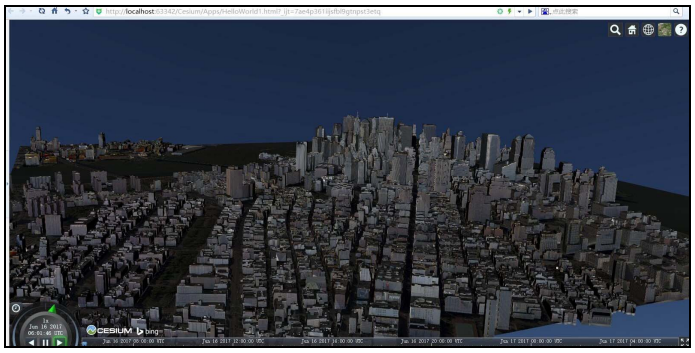
Fig. 8. Architecture of 3D roaming system.

The web server mainly provided data services that stored 3D scenes data in glTF formats. The client side was divided into the management layer and application layer, which were mainly responsible for scheduling and rendering of scenes data. When the scenes were rendered, the application layer made requests to the local cache for scene data. If no data for a task was located in the local cache, it made a request to the server. As the same time, the data request task was managed in queue by the management layer using the prefetching strategy. The client side could download some scenes in advance by predicting user trajectory. Finally, the scenes were rendered efficiently based on the visibility elimination strategy in the application layer. The 3D scenes visualization platform was developed based on Cesium. Cesium was developed by an open-source community, with support from the Cesium Consortium [14]. Cesium JS is a JS library for map rendering in the GIS industry, using WebGL for rendering maps. Using HTML5, rendering maps can be constructed in 3D.

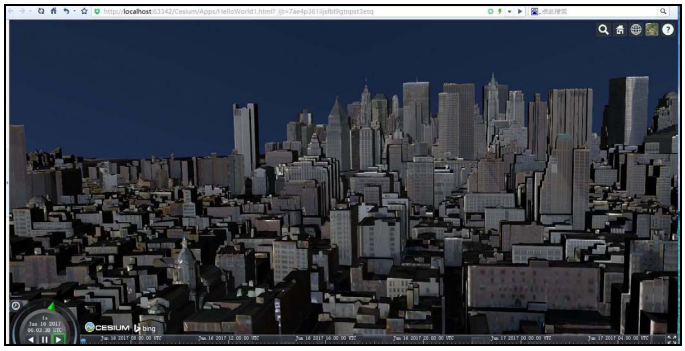
B. Experiment and Results

To prove the advantage of the proposed method based on system architecture, the 3D scene data of part of New York City was for experiments. This data included 765 models, 354,249 polygons, and 549,702 vertexes. There was 35.6 Mb disk space size taken up by the scene data before conversion, but this was reduced to 27.7 Mb after conversion to the glTF format. This suggests that the scene data based on glTF took up little space, and had a high compression ratio.

The server side hardware environment was an Intel E5-1650V4 6 cores 3.60 GHz CPU, and the memory was 16GB ECC. The video card was NVIDIA M4000, 8GB DDR5. The software environment was the Linux operating system and Tomcat server. The client used the Chrome Brower. The platform's first transmission response time was less than 3 s, and the average response time was less than 1 s. The average frame rate of scene browsing was 36 fps. The roaming speed of the platform was fast, and the picture rendering was fluent. The 3D scenes models and rendering of New York City are shown in Fig. 9.



(a) 3D buildings of New York City, USA



(b) Building details

Fig. 9. 3D scene of New York City, USA.

IV. CONCLUSION

Based on analysis of the Web 3D scene data transmission and visual characteristics, a new 3D model structure glTF was proposed in this paper. Since glTF is very suitable for Web exchange and rendering, a 3D scene organization and construction method was proposed based on glTF. We also designed a system service architecture based on the glTF format, prefetching strategy based on prediction trajectories, occlusion culling technology, and WebGL. Experimental results showed that the proposed system service framework could alleviate the pressure of bandwidth, and improve visualization efficiency.

ACKNOWLEDGMENT

This study was supported by the National Natural Science Foundation of China (No. 41501432; No. 41201411); National Special Program on Basic Works for Science and Technology of China (No. 2013FY110900); Foundation of State Key

Laboratory of Resources and Environmental Information System (No. O88RA20CYA).

REFERENCES

[1] G. Bell, A. Parisi and M. Pesce, "The Virtual Reality Modeling Language: Version 1.0 Specification", URL: http://vrml. wired. com/vrml. tech/vrml10-3. html, 1995.

[2] M. Reddy, L. Iverson, Y. G. Leclerc, "Under the hood of GeoVRML 1.0", Symposium on Virtual Reality Modeling Language, ACM, 2000, pp:23-28.

[3] M. Christen, S. Nebiker, and B. Loesch, "Web-Based Large-Scale 3D-Geovisualisation Using WebGL: The OpenWebGlobe Project", International Journal of 3-D Information Modeling, vol. 1(3), pp: 16-25, 2012.

[4] C. Marin, "WebGL Specification", Khronos WebGL Working Group, Retrieved from https://www.khronos.org/registry/webgl/specs/1.0/, 2011.

[5] R. Wüest, M. Christen, H. Eugster, "Processing and Rendering Massive 3D Geospatial Environments using WebGL", The Graphical Web, 2012.

[6] M. Mer, R. Gutbell, "A case study on 3D geospatial applications in the web using state-of-the-art WebGL frameworks", International Conference on 3d Web Technology, ACM, 2015, pp:189-197.

[7] W. J. Schroeder, J. A. Zarge, W. E. Lorense, "Decimation of triangle meshes", ACM Siggraph Computer Graphics, ACM, 1992, vol.26(2), pp: 65-70.

[8] V. Isler, R. W. H. Lau, W. H. Lau, "Real-time multi-resolution modeling for complex virtual environments", Symposium on virtual reality software and technology, 1996, pp: 11-20.

[9] H. Hoppe, T. DeRose, T. Duchamp, "Mesh optimization", Proceedings of the 20th annual conference on Computer graphics and interactive techniques, ACM, 1993, pp: 19-26.

[10] Fabrice Robinet, "glTF-the runtime asset format for WebGl, OpenGL ES, and OpenGL[DB/OL]", America:Fabrice Robinet, 2014, http://github.com/KhronosGroup/glTF/blob/master/specification/README.md.

[11] J. El-Sana, N.Sokolovsky, C. T. Silva, "Integrating occlusion culling with view-dependent rendering", Conference on Visualization, IEEE Computer Society, 2001, pp: 371-378.

[12] S. Coorg, S. Teller, "Real-time occlusion culling for models with large occluders", Symposium on Interactive 3d Graphics, ACM, 1997, pp: 83-ff.

[13] R. Kodituwakku, K. Wijeweera, M. Chamikara, "Anefficientline clipping algorithm for 3D space", International Journal of Advanced Research in Computer Science and Software Engineering, vol.2(5), pp: 96-101, 2012.

[14] U. D. Staso, M. Soave, A. Giori, "Heterogeneous-Resolution and Multi-Source Terrain Builder for CesiumJS WebGL Virtual Globe", Icvaiv 2016, International Conference on Visual Analytics and Information Visualisation, 2016.